PROJECT ACTIVITY REPORT

Activity 2.1: Face Detection and Tracking

Project: Multimodal Attendance & Engagement Agent

1. Activity Overview & Objective

This activity implements face detection and tracking to identify and monitor participants throughout a session. Video is captured from a live webcam or a pre-recorded classroom video, and each frame is processed sequentially to detect human faces with computer vision techniques.

Continuous identity and presence foundation. Once detected, each face is assigned a unique identifier that stays consistent across frames. Tracking logic associates detections in consecutive frames using spatial location and movement, so a participant remains tracked through slight head movements, orientation changes and brief occlusions.

Objectives

- Capture sequential frames from a webcam (`camera_id=0`) or a recorded video.
- Detect frontal faces with an OpenCV Haar Cascade on grayscale frames.
- Assign and maintain a unique participant ID per face across frames.
- Emit structured telemetry (ID, timestamp, bounding box) for presence-duration and engagement analysis.

2. Vision Pipeline & Tracking Architecture

The pipeline converts raw video frames into structured spatial telemetry containing participant identifiers, timestamps and bounding-box coordinates (`bbox`). This telemetry is the basis for presence-duration calculation and downstream engagement analysis.

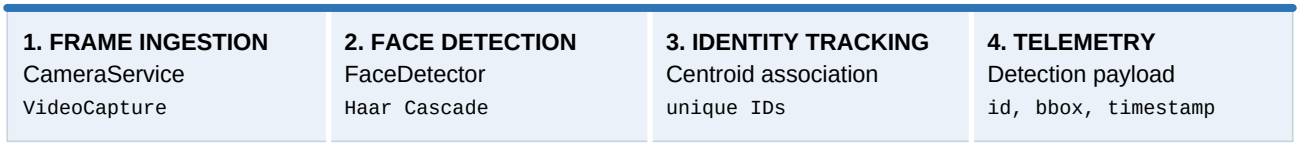

Figure 1: Four-stage face detection and tracking pipeline.

Pipeline stage	Technical mechanism	Operational role in classroom monitoring
Frame ingestion	OpenCV VideoCapture lifecycle managed by `CameraService`.	Captures sequential frames from a live webcam (camera_id=0) or a pre-recorded video stream.
Multi-scale face detection	Grayscale conversion (COLOR_BGR2GRAY) and Haar Cascade (haarcascade_frontalface_default.xml).	Scans frames with scaleFactor=1.1, minNeighbors=5, minSize=(30, 30) for cross-version stability.
Spatial identity tracking	Bounding-box centroid association (x, y, w, h) across consecutive frames.	Maintains unique participant IDs despite minor head turns, posture shifts or brief occlusions.
Structured telemetry output	Standardized detection dictionary with timestamps and positional tuples.	Feeds presence-duration calculators and multimodal engagement fusion layers.

3. Backend Implementation

The vision subsystem is split into two core services: `FaceDetector` (`backend/core/face_detection.py`) for frontal face localization, and `CameraService` (`backend/core/camera_service.py`) for managing the video stream. Section 4 adds the tracking module that turns per-frame detections into persistent participant IDs.

3.1 Face Detection Module

```
# File: backend/core/face_detection.py
import cv2


class FaceDetector:
    def __init__(self):
        # OpenCV's built-in Haar cascade: more robust across Python
        # versions than MediaPipe, and low-latency on CPU
        path = cv2.data.haarcascades + "haarcascade_frontalface_default.xml"
        self.face_cascade = cv2.CascadeClassifier(path)
        if self.face_cascade.empty():
            print("Error: Could not load face cascade xml")

    def process_frame(self, frame):
        # Convert to grayscale for detection
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

        # scaleFactor=1.1, minNeighbors=5, minSize=(30, 30)
        faces = self.face_cascade.detectMultiScale(gray, 1.1, 5, minSize=(30, 30))

        detections = []
        for (x, y, w, h) in faces:
            # bbox (x, y, w, h); score is a 1.0 placeholder
            detections.append({"bbox": (int(x), int(y), int(w), int(h)),
                               "score": 1.0})
        return detections
```

3.2 Video Stream Acquisition Service

```
# File: backend/core/camera_service.py
import cv2


class CameraService:
    def __init__(self, camera_id=0):
        self.camera_id = camera_id
        self.cap = None
        self.is_running = False

    def start(self):
        if self.cap is None:
            self.cap = cv2.VideoCapture(self.camera_id)
        self.is_running = True

    def stop(self):
        if self.cap:
            self.cap.release()
            self.cap = None
        self.is_running = False

    def get_frame(self):
        if self.is_running and self.cap:
            ret, frame = self.cap.read()
            if ret:
                return frame
        return None
```

For a pre-recorded classroom video, pass a file path to `cv2.VideoCapture` instead of a camera index (for example `CameraService("class.mp4")`); the rest of the pipeline is unchanged.

4. Identity Tracking Module

The detector alone only reports where faces are in each frame. The tracker below links detections across frames: it computes each bounding-box centroid, matches existing tracks to new detections by nearest distance (one-to-one, within `max_distance`), keeps unmatched tracks alive for `max_missed` frames to survive brief occlusions, and registers any unmatched detection as a new participant.

File: backend/core/face_tracker.py (reference implementation)

```
# File: backend/core/face_tracker.py
import math
import time


class CentroidTracker:
    """Associates face detections across frames using centroid distance."""

    def __init__(self, max_distance=80, max_missed=15):
        self.max_distance = max_distance   # max pixel jump between frames
        self.max_missed = max_missed       # frames a track may be unseen
        self.next_id = 1
        self.tracks = {}

    @staticmethod
    def _centroid(bbox):
        x, y, w, h = bbox
        return (x + w / 2.0, y + h / 2.0)

    def update(self, detections, timestamp=None):
        ts = time.time() if timestamp is None else timestamp
        cents = [self._centroid(d["bbox"]) for d in detections]
        # 1. Score every (track, detection) pair within range, nearest first
        pairs = []
        for tid, t in self.tracks.items():
            for j, c in enumerate(cents):
                dist = math.dist(t["centroid"], c)
                if dist <= self.max_distance:
                    pairs.append((dist, tid, j))
        pairs.sort()

        # 2. Greedy one-to-one matching
        used_tracks, used_dets = set(), set()
        for _, tid, j in pairs:
            if tid in used_tracks or j in used_dets:
                continue
            used_tracks.add(tid)
            used_dets.add(j)
            t = self.tracks[tid]
            t.update(bbox=detections[j]["bbox"], centroid=cents[j],
                     score=detections[j]["score"], missed=0, last_seen=ts)

        # 3. Age unmatched tracks; drop those missing for too long
```

```
        for tid in list(self.tracks):
            if tid not in used_tracks:
                self.tracks[tid]["missed"] += 1
                if self.tracks[tid]["missed"] > self.max_missed:
                    del self.tracks[tid]

        # 4. Register new participants
        for j, d in enumerate(detections):
            if j not in used_dets:
                self.tracks[self.next_id] = {
                    "bbox": d["bbox"], "centroid": cents[j],
                    "score": d["score"], "missed": 0,
                    "first_seen": ts, "last_seen": ts}
                self.next_id += 1

        # 5. Emit structured telemetry for currently visible faces
        return [{"id": tid, "bbox": t["bbox"], "score": t["score"],
                 "timestamp": ts,
                 "presence_sec": round(t["last_seen"] - t["first_seen"], 2)}
                for tid, t in self.tracks.items() if t["missed"] == 0]
```

Output telemetry

Each visible participant is emitted per frame in a standard schema, ready for presence-duration and engagement modules:

```
{"id": 1, "bbox": (110, 105, 50, 50), "score": 1.0,
 "timestamp": 1730000001.0, "presence_sec": 1.0}
```

Tracker behaviour check

The tracking logic was exercised with a synthetic sequence of two moving faces: IDs stayed stable while both faces moved slightly, a face hidden for one frame kept its ID when it reappeared, and a face absent for longer than `max_missed` frames was re-registered with a new ID. This test used synthetic bounding boxes only; it has not been run on real video.

5. End-to-End Pipeline Runner

This loop connects the three components and draws each tracked ID on the frame. Press q to quit.

```
# File: scripts/run_tracking.py
import time
import cv2
from backend.core.camera_service import CameraService
from backend.core.face_detection import FaceDetector
from backend.core.face_tracker import CentroidTracker


def run(source=0):
    cam, detector, tracker = CameraService(source), FaceDetector(), CentroidTracker()
    cam.start()
    try:
        while True:
            frame = cam.get_frame()
            if frame is None:
                break
            tracked = tracker.update(detector.process_frame(frame), time.time())
            for t in tracked:
                x, y, w, h = t["bbox"]
                cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 200, 0), 2)
                cv2.putText(frame, f"ID {t['id']}", (x, y - 8),
                            cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 200, 0), 2)
            cv2.imshow("Face Tracking", frame)
            if cv2.waitKey(1) & 0xFF == ord("q"):
                break
    finally:
        cam.stop()
        cv2.destroyAllWindows()


if __name__ == "__main__":
    run()
```

Run from the project root with the virtual environment active: `python -m scripts.run_tracking` (requires an `__init__.py` in `scripts/` and `backend/`), or add the project root to PYTHONPATH.

6. Parameter Tuning Guide

Parameter	Default	Effect	Tune when
scaleFactor	1.1	Image pyramid step. Smaller is more accurate but slower.	Small or distant faces are missed (try 1.05).
minNeighbors	5	Required neighbouring hits. Higher gives fewer false positives.	False boxes appear (raise) or real faces drop (lower).
minSize	(30, 30)	Smallest face detected, in pixels.	Back-row participants are missed (lower) or noise appears (raise).
max_distance	80 px	Max centroid jump to count as the same person.	IDs swap in crowded rows (lower) or fast movers get new IDs (raise).
max_missed	15 frames	Frames a track survives without a detection.	IDs reset after short occlusions (raise) or ghost tracks linger (lower).

7. Key Engineering Highlights

- **Cross-environment reliability:** OpenCV's bundled `haarcascade_frontalface_default.xml` avoids third-party binary dependency conflicts across Python environments and keeps CPU latency low.

- **Standardized bounding-box schema:** emitting `{"bbox": (x, y, w, h), "score": 1.0}` records enables spatial association across frames for persistent IDs and presence-duration computation.
- **Modular services:** capture, detection and tracking are independent classes, so any one can be replaced (for example a DNN detector) without touching the others.
- **Occlusion tolerance:** the missed-frame grace period keeps IDs stable through brief blockages.

8. Known Limitations & Future Improvements

Limitation	Impact	Possible improvement
Haar cascade detects frontal faces only	Turned or looking-down faces are missed, which can look like absence.	Add a profile cascade or move to a DNN detector (for example YuNet or MediaPipe).
Score is a fixed 1.0 placeholder	No confidence-based filtering.	Use a detector that returns real confidence values.
Centroid matching ignores appearance	IDs can swap when faces cross or sit very close.	Use IoU matching, motion prediction (Kalman filter) or face embeddings.
Tracks reset when a person leaves and returns after max_missed	Presence time splits across IDs.	Re-identify with face recognition to merge identities.
Sensitive to lighting and low resolution	Detection drops in dark or distant scenes.	Histogram equalization, better camera placement, higher resolution.

9. Validation Checklist

Tick each item after running it on your machine:

Check	How to verify	Status
Cascade file loads	FaceDetector() prints no error	[ ]
Webcam frames received	get_frame() returns a frame (not None)	[ ]
Faces detected	Green boxes appear on visible faces	[ ]
IDs persist while moving	Same ID label during small head movements	[ ]
Brief occlusion handled	ID returns after covering face for 1-2 seconds	[ ]
Video file input works	Run with a .mp4 path as the source	[ ]
Telemetry schema correct	Printed output has id, bbox, score, timestamp	[ ]